Universal oblong filter

Wednesday, April 20, 2022 10:36 PM

Before we formulate the algorithm, let's first derive the expression for the 1st – 4th order derivatives for an arbitrary point (a, b), in a coordinate system (x, y).

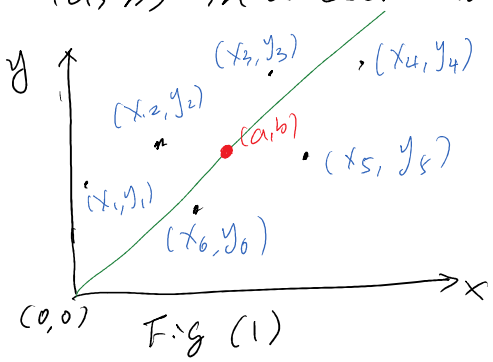


Fig (1)

As shown in the Fig (1) above, the line connecting origin $(0,0)$ and (a, b) does not go through any (x_i, y_i), where (x_i, y_i) is the location of pixel center and "i" represents the "ith" pixel surrounding (a, b).

The Taylor expansion at (a, b) can be written as

$$f(x,y) = f(a,b) + f_x(a,b)(x-a) + f_y(a,b)(y-b) \quad \text{0th and 1st-order terms}$$

$$+ \frac{f_{xx}(a,b)}{2}(x-a)^2 + f_{xy}(a,b)(x-a)(y-b) + \frac{f_{yy}(a,b)}{2}(y-b)^2 \quad \text{2nd order terms}$$

$$+ \frac{f_{xxx}}{6}(x-a)^3 + \frac{f_{yyy}}{6}(y-b)^3 + \frac{f_{xxy}}{2}(x-a)^2(y-b) + \frac{f_{xyy}}{2}(x-a)(y-b)^2 \quad \text{3rd order}$$

$$+ \frac{f_{xxxx}}{24}(x-a)^4 + \frac{f_{yyyy}}{24}(y-b)^4 + \frac{f_{xxxy}}{6}(x-a)^3(y-b) + \frac{f_{xxyy}}{4}(x-a)^2(y-b)^2 + \frac{f_{xyyy}}{6}(x-a)(y-b)^3 \quad (1)$$

4th order

For shorthand notation, $f(a,b)$ will be denoted by f°

$f_{xxxy}(a,b)$ will be denoted by $f^\circ_{x^3y}$ and likewise.

Then Eq. (1) above can be rewritten as

$$f(x,y) = f^\circ + f^\circ_x(x-a) + f^\circ_y(y-b) + \frac{f^\circ_{x^2}}{2}(x-a)^2 + \frac{f^\circ_{y^2}}{2}(y-b)^2 + f^\circ_{xy}(x-a)(y-b)$$

$$+ \frac{f^\circ_{x^3}}{6}(x-a)^3 + \frac{f^\circ_{y^3}}{6}(y-b)^3 + \frac{f^\circ_{x^2y}}{2}(x-a)^2(y-b) + \frac{f^\circ_{xy^2}}{2}(x-a)(y-b)^2$$

$$+ \frac{f^\circ_{x^4}}{}(x-a)^4 + \frac{f^\circ_{y^4}}{}(y-b)^4 + f^\circ_{x^3y}(x-a)^3(y-b) + f^\circ_{x^2y^2}(x-a)^2(y-b)^2 + f^\circ_{xy^3}(x-a)(y-b)^3 \quad (2)$$

$\frac{1}{24}$ $\frac{1}{24}$ $\frac{1}{6}$ $\frac{1}{4}$ $\frac{1}{6}$

Note that Eq.(2) includes 0^{th} – 4^{th} order derivatives, and the total number of them is 15.

Then we substitute the pixels (x_i, y_i) surrounding (a,b) into Eq.(2)

$$f(x_i,y_i) = f^\circ + f_x^\circ (x_i-a) + f_y^\circ (y_i-b) + \frac{f_{x^2}^\circ}{2}(x_i-a)^2 + \frac{f_{y^2}^\circ}{2}(y_i-b)^2 + f_{xy}^\circ (x_i-a)(y_i-b)$$
$$+ \frac{f_{x^3}^\circ}{6}(x_i-a)^3 + \frac{f_{y^3}^\circ}{6}(y_i-b)^3 + \frac{f_{x^2y}^\circ}{2}(x_i-a)^2(y_i-b) + \frac{f_{xy^2}^\circ}{2}(x_i-a)(y_i-b)^2$$
$$+ \frac{f_{x^4}^\circ}{24}(x_i-a)^4 + \frac{f_{y^4}^\circ}{24}(y_i-b)^4 + \frac{f_{x^3y}^\circ}{6}(x_i-a)^3(y_i-b) + \frac{f_{x^2y^2}^\circ}{4}(x_i-a)^2(y_i-b)^2 + \frac{f_{xy^3}^\circ}{6}(x_i-a)(y_i-b)^3 \quad (3)$$

In Eq(3) all terms of (x_i-a) and (y_i-b) are known, because they are the positions of neighbouring pixels and (a,b). Thus, we have 15 unknowns, which are the 0th – 4th order derivatives. To obtain these 15 unknowns, we would need at least 15 pixels surrounding (a,b), i.e, $(x_1,y_1)\ (x_2,y_2)\ \cdots\ (x_{15},y_{15})$ $i=1,\ldots 15$

Then we will have 15 equations according to Eq(3) They can be cast into a compact matrix form

$$\underbrace{\begin{bmatrix} 1 & (x_1-a) & (y_1-b) & \frac{(x_1-a)^2}{2} & \frac{(y_1-b)^2}{2} & (x_1-a)(y_1-b) & \cdots & \frac{1}{4}(x_1-a)^2(y_1-b)^2 & \frac{1}{6}(x_1-a)(y_1-b)^3 \\ 1 & (x_2-a) & (y_2-b) & \frac{(x_2-a)^2}{2} & \frac{(y_2-b)^2}{2} & (x_2-a)(y_2-b) & \cdots & \frac{1}{4}(x_2-a)^2(y_2-b)^2 & \frac{1}{6}(x_2-a)(y_2-b)^3 \\ \vdots & & & & & & & & \\ 1 & (x_{15}-a) & (y_{15}-b) & \frac{(x_{15}-a)^2}{2} & \frac{(y_{15}-b)^2}{2} & (x_{15}-a)(y_{15}-b) & \cdots & \frac{1}{4}(x_{15}-a)^2(y_{15}-b)^2 & \frac{1}{6}(x_{15}-a)(y_{15}-b)^3 \end{bmatrix}}_{A} \underbrace{\begin{bmatrix} f^\circ \\ f_x^\circ \\ f_y^\circ \\ f_{x^2}^\circ \\ f_{y^2}^\circ \\ f_{xy}^\circ \\ \vdots \\ f_{x^2y^2}^\circ \\ f_{xy^3}^\circ \end{bmatrix}}_{z} = \underbrace{\begin{bmatrix} f(x_1,y_1) \\ f(x_2,y_2) \\ \vdots \\ f(x_{15},y_{15}) \end{bmatrix}}_{b} \quad (4)$$

Eq.(4): $A \in R^{15\times 15}$, $z \in R^{15}$ and $b \in R^{15}$. However if use more than 15 pixels. Then $A \in R^{N\times 15}$, $z \in R^{15}$ and $b \in R^{N}$, where N is the number of pixels surrounding (a,b). Then Eq.(4)

becomes a least squares problem. and can be solved using pseudo-inverse. i.e.

$$Z = (A^T A)^{-1} A^T \cdot b = A^* \cdot b. \qquad (5)$$

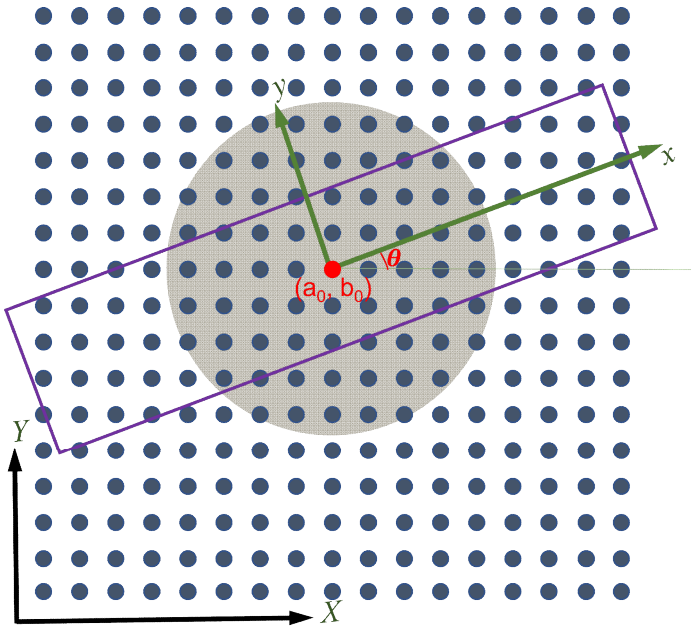
Now we have fundamentals to calculate 0th-4th order derivatives at any location and the surrounding pixels don't need to be along the horizontal or the vertical axes. Next I will apply it to the oblong filter.

The oblong filter along an arbitrary angle θ is shown on the right. The origin (a_0, b_0) is on a pixel.

(X, Y) is the global image coordinate

(x, y) is the local coordinate

The yellow region is range of the surrounding pixels. That is, there are N pixels in the yellow region that surround the origin (a_0, b_0)



In the global coordinate system, these surrounding pixels are

$$(X_1, Y_1), (X_2, Y_2), \cdots (X_i, Y_i), \cdots (X_N, Y_N); \quad i = 1, 2, \cdots N.$$

First, we will convert these N surrounding pixels to the local coordinate system (x, y) associated with the oblong filter along θ direction, which is given by

$$\begin{bmatrix} x_i \\ y_i \end{bmatrix} = \begin{bmatrix} \cos\theta & \sin\theta \\ -\sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} X_i - a_0 \\ Y_i - b_0 \end{bmatrix} \quad i = 1, 2, \cdots N \qquad (5.5)$$

Then we apply Eq. (4) to the origin. However, at origin. (a, b) in Eq. (4) will be $(0, 0)$. Then Eq. (4) for the pixel

(a, b) in Eq. (4) will be (0, 0). Then Eq. (4) for the pixel at the origin. will be.

$$\begin{bmatrix} 1 & x_1 & y_1 & \frac{x_1^2}{2} & \frac{y_1^2}{2} & x_1y_1 & \cdots & \frac{1}{4}x_1^2y_1^2 & \frac{1}{6}x_1y_1^3 \\ 1 & x_2 & y_2 & \frac{x_2^2}{2} & \frac{y_2^2}{2} & x_2y_2 & \cdots & \frac{1}{4}x_2^2y_2^2 & \frac{1}{6}x_2y_2^3 \\ \vdots & & & & & & & & \\ 1 & x_N & y_N & \frac{x_N^2}{2} & \frac{y_N^2}{2} & x_Ny_N & \cdots & \frac{1}{4}x_N^2y_N^2 & \frac{1}{6}x_Ny_N^3 \end{bmatrix} \begin{bmatrix} f^0 \\ f_x^0 \\ f_y^0 \\ \vdots \\ f^0_{x^2y^2} \\ f^0_{xy^3} \end{bmatrix} = \begin{bmatrix} f(x_1, y_1) \\ f(x_2, y_2) \\ \vdots \\ f(x_N, y_N) \end{bmatrix} \quad (6)$$

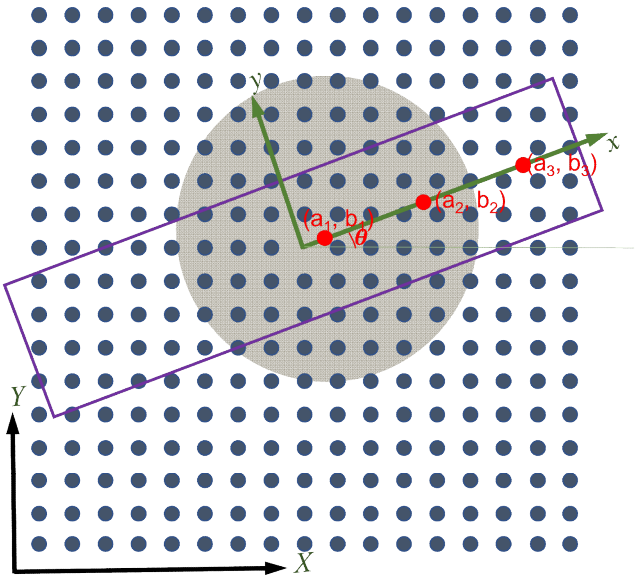
A_0 Z_0 b_0

Here subscript "0" denotes "at origin" Then 0th-4th order derivative is written as

$$Z_0 = (A_0^T A_0)^{-1} A_0^T \cdot b_0 = A_0^{\dagger} b_0 \quad (7)$$

Next we will calculate the derivatives at several locations (a_1, b_1) $(a_2, b_2) \ldots (a_m, b_m)$.

Here m is number of points used to resolve the half length of the filter. Again the yellow region represents the range of surrounding pixels.



In addition, since $(a_1, b_1) \ldots (a_m, b_m)$ along x axis of the local coordinate, they can be simplified as $(a_1, 0), (a_2, 0) \ldots (a_m, 0)$

Select any point $(a_j, 0)$ $j = 1, 2, \ldots m$ as an example we first find N sourrending pixels $(X_1, Y_1), \ldots (X_N, Y_N)$ convert them to the local coordinate system Using Eq. (5.5)

to obtain $(x_1, y_1)\ (x_2, y_2)\ \cdots\ (x_N, y_N)$. Then Eq. (4) at location $(a_j, 0)$ is given by

$$\begin{bmatrix} 1 & (x_1-a_j) & y_1 & \frac{(x_1-a_j)^2}{2} & \frac{y_1^2}{2} & \cdots & \frac{1}{4}(x_1-a_j)^2 y_1^2 & \frac{1}{6}(x_1-a_j) y_1^3 \\ 1 & (x_2-a_j) & y_2 & \frac{(x_2-a_j)^2}{2} & \frac{y_2^2}{2} & \cdots & \frac{1}{4}(x_2-a_j)^2 y_2^2 & \frac{1}{6}(x_2-a_j) y_2^3 \\ \vdots & & & & & & & \\ 1 & (x_N-a_j) & y_N & \frac{(x_N-a_j)^2}{2} & \frac{y_N^2}{2} & \cdots & \frac{1}{4}(x_N-a_j)^2 y_N^2 & \frac{1}{6}(x_N-a_j) y_N^3 \end{bmatrix} \begin{bmatrix} f^0 \\ f_x^0 \\ f_y^0 \\ \vdots \\ f_{x^2y^2}^0 \\ f_{xy^3}^0 \end{bmatrix} = \begin{bmatrix} f(x_1, y_1) \\ f(x_2, y_2) \\ \vdots \\ f(x_N, y_N) \end{bmatrix}$$

A_j $\quad$ z_j $\quad$ b_j (8)

Then the 0th-4th order derivatives at $(a_j, 0)$ location within the local coordinate system is given by

$$z_j = (A_j^+ A_j)^{-1} A_j^+ b_j = A_j^* b_j \qquad (9)$$

we can repeat such calculation at a_j points. i.e., z_j

Finally all these z_j will be weight-averaged to obtain the value at the origin of the local coordinate system as we did for the horizontal, vertical and 45-135-deg direction. i.e.,

$$\tilde{z}_0 = \sum_{j=-m}^{m} w_j \cdot z_j \qquad (10)$$

where w_j is the weight applied at $(a_j, 0)$ to caltion.

With the formulation above, next everything will be put together for "Numerical Implementation". The entire process consists of

two stages: (1) building the universal edge filter and
(2) applying/using the filter

For the first stage to develop the filter, we outlined

1. for $l = 0 : 1 : \frac{2\pi}{\Delta\theta}$ // $\Delta\theta$ angle resolution along circumferential direction
$\theta = l \cdot \Delta\theta$
for $j = 0 : 1 : 2m$ // grid points $(a_j, 0)$ above along radial direction
$K_j = knn(a_j, 0)$ // find N surrounding pixels around a_j.
K_j is the indices of these N pixels.
$N = length(K_j)$
$(X_i, Y_i) = global\_coordinate(K_j)$; // get global coordinates of the N pixels.
$(x_i, y_i) = local\_coordinate(X_i, Y_i, \theta)$; // get local coordinates of the N pixels using Eq. (5.5)
$A_j = assemble\_A_j(x_i, y_i, a_j)$; // Assemble A_j matrix in Eq(6)
$A_j^* = pinv(A_j)$ // calculate pseudo-inverse of A_j
(Note that not every entry in z is needed, for example we only need $\frac{\partial f}{\partial y}$, $\frac{\partial^2 f}{\partial y^2}$, $\frac{\partial^3 f}{\partial y^3}$ and $\frac{\partial^4 f}{\partial y^4}$. To reduce the memory usage, we can only save the rows of A_j^* for these entries., that is)
$A_j^* = A_j^*(q, :)$; // only extract rows of A_j^* corresponding to q indices. q is indices corresponding to the derivatives of our interest.
$filter\{l\}.gridpoint\{j\}.weights = A_j^*$
// Save the weights in a structure array.
end for
end for.

At the end of filter construction, we will have
where r_0 is the resolution of

end for.

* At the end of filter construction, we will have $\left(\frac{2\pi}{\Delta\theta}\right) \cdot (2m+1)$ sets of weights, where $\Delta\theta$ is the resolution of the orientation angles. $(2m+1)$ is the number of grid points along the longitudinal direction.

The next stage is to use these sets of filter weights to calculate the 1st - 4th order derivatives at any pixel location (a_0, b_0)

The algorithm is given below.

```
for a0 = 1:1:NH        // NH and NV are the number of pixel
  for b0 = 1:1:NV        along horizontal and vertical direction
                         applicable to derivative calculation
    for l = 1:1:2π/Δθ
      for j = 1:1:2m+1
        Kj = knn(aj, 0);   // extract indices of
                              the neighbouring
                              pixels
        bj = [ f1 ; f2 ; ... ; fN ]   // collect intensity values of the
                                         N neighbouring pixels using
                                         index Kj
        Aj* = filter{l}.gridpoint{j}.weights ;
                // extract filter weights at j
                // grid point
        Zj = Aj* . bj   // calculate all derivatives.
      end for (j loop).
      Zavg(a0,b0,l) = sum(Zj . Wj);   // Wj is the weights of derivatives
                                         in Eq. (10)
    end for (l loop)
  end for (b0 loop)
```

end for (a0 loop)

* It should be noted that in this approach, we will calculate all derivatives (0th to 4th-order) in one shot. (in both normal and tangential direction)

- Another challenge is if we can avoid using "knn" function in Matlab to find N nearest surrounding pixels. We should be able to. During the filter development stage, at each @ and each gridpoint (j) location, we will use "knn" to find the N nearest pixels. Then we save their relative pixel distances from the origin pixel (a_0, b_0). For example,

For $j=0$, we have N $(X_1, Y_1), (X_2 Y_2), \cdots (X_N, Y_N)$

the the relative indices for $j=0$ will be

$(X_i^{j=0}, Y_i^{j=0}) - (a_0, b_0)$, which will be saved

we then repeat the same for $j=1$ and get

$(X_i^{j=1}, Y_i^{j=1}) - (a_0, b_0)$ will will be saved.

End

4/24/2022 Yi Wang @ UofSC